

Unit 02: Analysis of Algorithm And Complexity theory

Page No.	
Date	

- a) time complexity $\Rightarrow$ No. of iteration
- b) space complexity $\Rightarrow$ Memory consumption

* Analysis

i) Input size

i) Input size is the number of elements in the input.

ii) Running

ii) The measure of time efficiency as a function of input size.

iv) Different input of size 'n' may take different amount of time.

2) How to determine the input size

i) Natural parameter

ii) For sorting or other problem an array input size depend on array size.

iii) For combinational problem - number of object

iv) For Graph - no. of vertices and Edges.

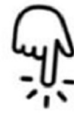
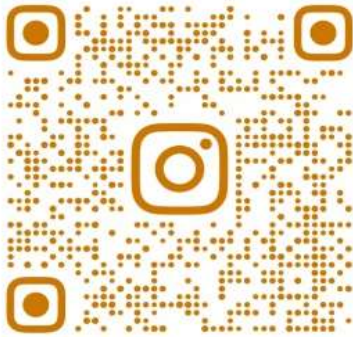
So, Important is class or problem.

* Types of Analysis

i) The function $f(n)$, gives the running of an algorithm depends not only on the size (n) of the input data but also on the particular data.

ii) The complexity function $f(n)$ for certain cases are,

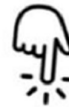
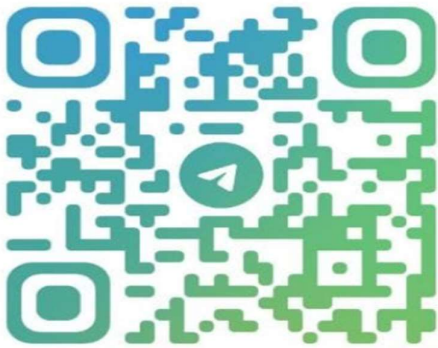
Join community by clicking below links



Insta Page

https://www.instagram.com/sppu_engineering_update

@SPPU_ENGINEERING_UPDATE



Telegram Channel

https://t.me/SPPU_TE_BE_COMP

@SPPU_TE_BE_COMP



WhatsApp Channel

<https://whatsapp.com/channel/0029VaAhRMdAzNbmPG0jyq2x>

a) Best case

- i) The minimum possible values of $T(n)$ is called the Base case.
- ii) Best case complexity of the algorithm is the function defined by the minimum number of steps taken on any instance or size n .
- iii) It provides a lower bound on running time.
- iv) Input is the one for which the algorithm run is the fastest.

b) Average case

- i) The expected value of $T(n)$
- ii) Average case complexity of the algorithm is the function defined by the average number of steps taken on any instance of size n .
- iii) It provides a prediction about running time and assumes that input is random.

c) Worst case

- i) Worst case complexity of the algorithm is the function defined by the maximum no. of steps taken on any instance of size n .
- ii) It provides the upper bound on running time.
- iii) An absolute Guarantee that the algorithm would not run longer than worst case no matter what the input are.

que-1) What is Best, Average and worst case Analysis of Algorithm & Analysis following algorithm Best, Average and worst case:

Ex: int LinearSearch (int a, int n, int item)

```
{  
    int i;  
    for (i=0; i<n; i++)  
    {  
        if (a[i] == item) then  
        {  
            return a[i];  
        }  
    }  
}
```

return -1;

⇒ For above algorithm,

a) Best case

Best case is $O(1)$ this is a case when the item to be searched in a array found at first position.

b) Worst case

i) worst case time complexity of above algorithm is $O(n)$

ii) Here, worst case means the item to be searched in the given array is found at last position or not found at all.

c) Average case

i) Average case time complexity of above algo. is $O(n)$

ii) The average case means the item is searched is found at the middle position or nearer position of the array.

iii) As per this statement the time complexity will be $O(n/2)$ but we cannot write $O(n/2)$

iv) so, instead of that we write down as $O(n)$.

QUP-2) What is Best, Average and worst case analysis Algorithm & Analyze the following algorithm Best, Average and worst case?

```
void sort (int a, int n)
```

```
{  
    int i, j;
```

```
    for (i = 0; i < n; i++)
```

```
    {  
        j = i - 1;
```

```
        key = a[i];
```

```
        while (j >= 0 && a[j] > key)
```

```
        {  
            a[j+1] = a[j];
```

```
            j = j - 1;
```

```
        }  
        a[j+1] = key;
```

```
    }
```


→ a) Best Case

- i) Best case time complexity of above algorithm is $O(n)$.
- ii) Best case is when the given array is already sorted, in that case the inner while loop will iterate only one in a while loop.
- iii) So, time complexity completely depend on for loop and as a for loop is iterating n times.
- iv) So, the Best case time complexity here is $O(n)$.

b) Worst Case

- i) When we want to sort array in ascending order but given array is in descending order in that case the inner while loop will iterate maximum n time and outer for loop will also iterate till n time.
- ii) So, the worst case time complexity will be $n \times n$ i.e. $O(n^2)$.

c) Average Case

- i) In Average case, the array is having random input.
- ii) So, in this case outer loop will iterate n time and inner loop will iterate $n/2$ time.
- iii) So, the time complexity will be $n \times \frac{n}{2} =$

$$\frac{n^2}{2} \text{ which is equal to } O(n^2)$$

* Asymptotic Growth

- i) it is a mathematical way of representing the time complexity.
- ii) To compare two algorithms with running times $f(n)$ and $g(n)$ we need a rough measure that characterizes how fast each function grows.
- iii) Unless we evaluate the performance of an algorithm by taking a sufficiently large input size, we can justify its growth rate.
- iv) hence, we study asymptotic growth of an algorithm where a performance of an algorithm verified against the increasingly input size bound lessely.

* Types of Asymptotic notation

- 1) Big "oh" (O)
- 2) Big omega (Ω)
- 3) Big Theta (Θ)
- 4) Little oh (o)
- 5) Little omega (ω)

1) Big "oh" (O)

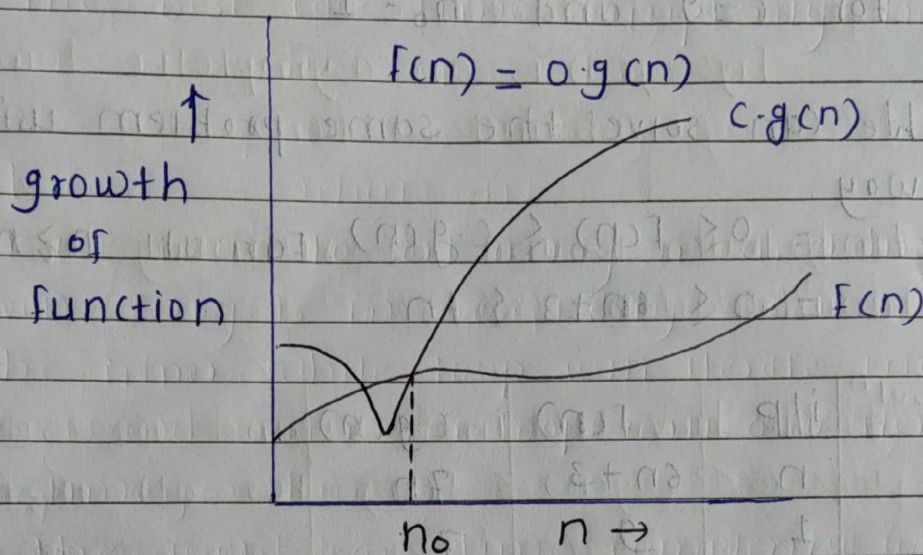
- i) This notation is denoted by O and it is pronounced as Big "oh".
- ii) "Big oh" notation defines upper bound for the algorithm. It means that running time of algorithm cannot be more than its asymptotic upper bound for any n data.
- iii) Let, $f(n)$ and $g(n)$ are two non-negative function indicating running time of two

algorithm.

iv) We say $g(n)$ is upper bound of $f(n)$, if there exist some positive constant 'c' and 'n₀' such that

$$0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0$$

It is denoted as



$g(n)$ is an asymptotic upper bound for $f(n)$

Que. 1) Find upper bound of running time of Linear function $f(n) = 6n + 3$

→ Soln: We have to find value of c & n₀ such that,

$$0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0$$

$$= 0 \leq 6n + 3 \leq 6n + 3n$$

$$= 0 \leq 6n + 3 \leq 9n$$

i/p	$f(n)$	$c \cdot g(n)$
n	$6n+3$	$9n$
1	9	9
2	15	18
3	21	27
4	27	36

Same value so, $n_0 = 1$

From above table we can say that $f(n) = O(g(n))$ for $c=9$ and $n_0=1$

We can solve the same problem using another way.

$$0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0$$

$$0 \leq 6n+3 \leq 7n$$

i/p	$f(n)$	$c \cdot g(n)$
n	$6n+3$	$7n$
1	9	7
2	15	14
3	21	21
4	27	28

$\Rightarrow f(n) = O(g(n))$ so, $n_0 = 3$

From above table we can say that $f(n) = O(g(n))$ for $c=7$ and $n_0=3$

Que. 2) Find upper bound of $f(n) = 3n^2 + 2n + 4$

$\Rightarrow$ Soln: We have to find value of c and n_0 such that,

$$= 0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0$$

$$= 0 \leq 3n^2 + 2n + 4 \leq 3n^2 + 2n^2 + 4n^2$$

$$= 0 \leq 3n^2 + 2n + 4 \leq 9n^2$$

i/p	f(n)	c.g(n)
n	$3n^2 + 2n + 4$	$9n^2$
1	9	9
2	20	36
3	37	81
4	60	144

From above table we can say that $f(n) = O(g(n))$
For $c=9$ and $n_0=1$

2) Big omega notation (Ω)

- This notation is denoted by ' Ω ' symbol and it is pronounced as Big omega.
- Big omega defines lower bound for the algorithm.
- It means running time of algorithm cannot be less than its asymptotic lower bound for any random sequence of data.
- We say that $g(n)$ is lower bound of $f(n)$ if there exist some positive constant ' c ' and ' n_0 ' such that,

$$0 \leq c \cdot g(n) \leq f(n) \text{ for all } n \geq n_0$$

- It is denoted as $f(n) = \Omega(g(n))$

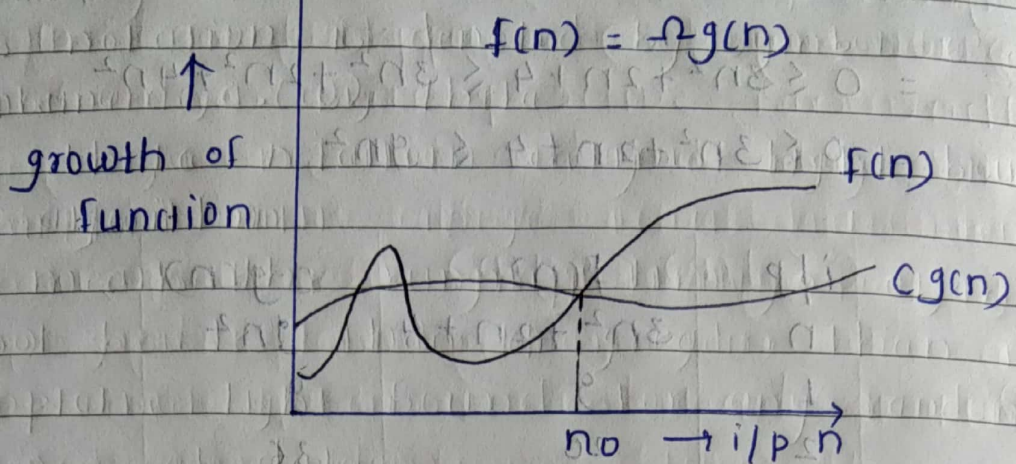


Fig: $g(n)$ is an asymptotic lower bound of $f(n)$

Ques.1) Find lower bound of running time of linear function $f(n) = 6n + 3$

$\rightarrow$ Solⁿ: We have to find c and n_0 such that $0 \leq c \cdot g(n) \leq f(n)$ for all $n \geq n_0$

$$= 0 \leq c \cdot g(n) \leq 6n + 3$$

$$= 0 \leq 6n \leq 6n + 3$$

i/p	$f(n)$	$c \cdot g(n)$
n	$6n + 3$	$6n$
1	9	6 $\therefore 9 > 6$
2	15	12
3	21	18

From above table we can say that $f(n) = \Omega(n)$ for $c = 6$ and $n_0 = 1$

que. 2) Find lower bound of running time of quadratic function $f(n) = 3n^2 + 2n + 4$

→ soln: We have to find c and n_0 such that $0 \leq c \cdot g(n) \leq f(n)$ for all $n \geq n_0$

$$= 0 \leq c \cdot g(n) \leq 3n^2 + 2n + 4$$

$$= 0 \leq 3n^2 \leq 3n^2 + 2n + 4$$

i/p	$f(n)$	$c \cdot g(n)$
n	$3n^2 + 2n + 4$	$3n^2$
1	9	3
2	20	12

From above table we can say that $f(n) = \Omega(g(n))$

For $c=3$ and $n_0=1$

3) $f(n) = 3n^2$

→ soln: $0 \leq c \cdot g(n) \leq f(n)$

$$= 0 \leq c \cdot g(n) \leq 3n^2$$

$$= 0 \leq 2n^2 \leq 3n^2$$

i/p	$f(n)$	$c \cdot g(n)$
n	$3n^2$	$2n^2$
1	3	2
2	12	8

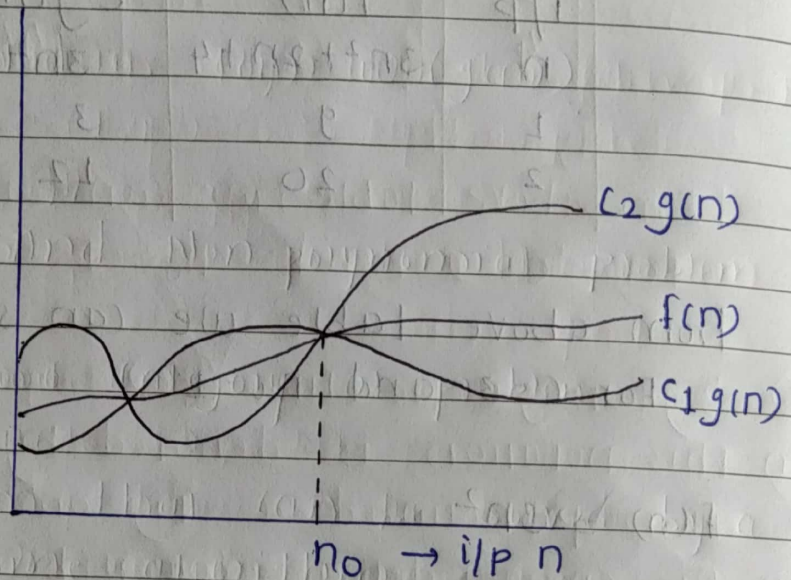
From above table we can say that $f(n) = \Omega(g(n))$

For $c=2$ and $n_0=1$

3) Big Theta (Θ)

- i) This notation is denoted by symbol ' Θ ' and is pronounced as Big theta.
- ii) Big theta notation defines tight bound for the algorithm.
- iii) It means the running time of algorithm cannot be less than or greater than its asymptotic tight bound for any random sequence of data.

↑
growth of
function



- iv) We say $g(n)$ is tight bound of $f(n)$ if there exist some positive constant c_1, c_2 and n_0 such that,

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0$$

- v) It is denoted as $f(n) = \Theta(g(n))$

que.1) Find tight bound of running time of constant function $f(n) = 23$

→ Soln: $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$

$$= 0 \leq c_1 g(n) \leq 23 \leq c_2 g(n)$$

$$= 0 \leq 22 \leq 23 \leq 24$$

$\therefore f(n) = \Theta(g(n))$ for $c_1 = 22$, $c_2 = 24$ and $n_0 = 1$

2) $f(n) = 6n + 3$

→ Soln: $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$

$$= 0 \leq 6n \leq 6n + 3 \leq 6n + 3n$$

$$= 0 \leq 6n \leq 6n + 3 \leq 9n$$

i/p	$c_1 g(n)$	$f(n)$	$c_2 g(n)$
n	$6n$	$6n + 3$	$9n$
1	6	9	9
2	12	15	18

From above table we can say that $f(n) = \Theta(g(n))$
for $c_1 = 6$, $c_2 = 9$, $n_0 = 1$

4) Little 'oh' (o)

i) Big oh notation may or may not be tight. It is also called loose upper bound. So little oh is used to defined bound which is asymptotically not tight.

ii) $f(n)$ grows slower than $g(n)$

iii) if $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$

Formally stated as $f(n) = o(g(n))$

5) Little omega (ω)

i) Big omega notation may or may not be tight but little omega is used to define bound which is asymptotically not tight.

ii) $f(n)$ grows faster than $g(n)$

iii) $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$

Formally stated as $f(n) = \omega(g(n))$

* polynomial And Non-polynomial problem

there are two category of problem

i) polynomial

i) problem that can be solved by a polynomial time algorithm

ii) Example: Searching - $O(\log n)$

Sorting - $O(n \log n)$

2) Non-polynomial

i) problems for which non-polynomial time algorithm is known.

ii) Example: a) Travelling salesman - $O(n^2 n)$

b) knapsack problem - $O(2^{n/2})$

Deterministic And Non-deterministic Algorithm

1) Deterministic Algorithm

i) Result of every operation is uniquely defined for a particular input computer will give always same output.

ii) Such algorithm can solve problem in polynomial time and can determine the next step of execution.

iii) Example $5+3=8$

iv) Above eqⁿ will produce always same output.

2) Non-deterministic Algorithm

i) For same input compiler may produce different output in different runs.

ii) Such algorithm can not solve problem in polynomial time and cannot determine the next step of execution due to more than one path the algorithm can follow.

iii) Example: $a+b=8$

iv) Here, the value of $a+b$ can have many possibilities like $1+7$, $6+2$, etc.

v) To specify non-deterministic algorithm, some terms are defined as follows:

i) choice(x)

It chooses any value randomly from set (x).

2) Failure ()
it denotes the unsuccessful solution

3) Success ()
it denotes a successful solution

Example: Search an element x on $A[1 \dots n]$,
where $n > 1$, on successful search return j ,
if $A[j] = x$ otherwise return 0.

Non deterministic algorithm for above problem

is, $j = \text{choice}(1, n)$
if $(A[j] = x)$
then

{
write (j);

Success ();

} write (0);

Failure ();

⇒

* Decision problem and optimization problem

i) Decision problem

i) Any problem for which the answer is either 0, 1 [yes or no] is called a decision problem

ii) An algorithm for a decision problem is

termed as decision algorithm

2) Optimization problem

- i) Any problem that involves the identification of an optimal solution (either maximum or minimum) of a given function is known as a optimization problem.
- ii) An optimization algorithm is used to solve an optimization problem.
- iii) Decision problem is assumed to be easier to solve compare to the optimization problem.

mp * P, NP, NP-hard, NP-complete

1) Definition of P

problem that can be solved in polynomial time using deterministic algorithm (P stands for polynomial)

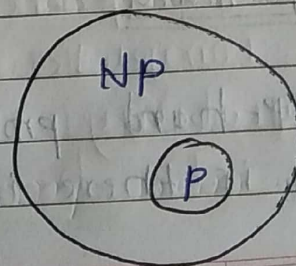
Example:

Searching, Sorting

2) NP

it stands non-deterministic polynomial time. it is a collection of decision problem that can be solved by a non-deterministic machine in polynomial time.

Traveling Salesman



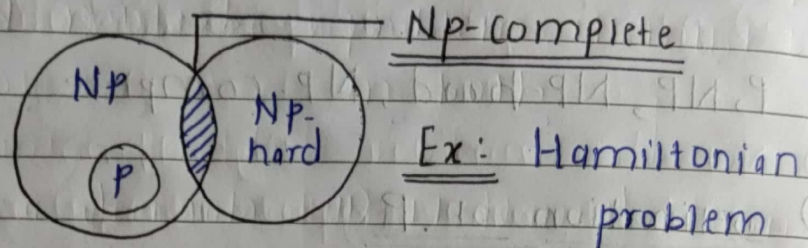
iii) since, deterministic algorithm are just a special case of non-deterministic one,

$$P \subseteq NP$$

iv) The NP class problem can be further categorized into NP-complete and NP-hard problem

3) NP-complete

i) A problem 'A' is called NP complete if



a) If it belongs to class NP

b) Every other problem 'B' in NP can be transformed to 'A' in polynomial time

ii) These two factors prove that NP-complete problems are the harder problems in class NP.

ii) However a solution of any NP-complete problem can be verified in polynomial time, but cannot be obtained in polynomial time

4) NP-hard

i) NP-hard problem (say A) can be solved if and only if there is NP-complete problem (say B)

can be reducible into 'A' in polynomial time

ii) Example:

- i) Halting problem
- ii) Hamiltonian cycle problem

* Difference between P & NP

P	NP
i) P problems are set of problems which can be solved in polynomial time by deterministic algorithm	i) NP problems are the problems which can be solved in non-deterministic polynomial
ii) P problems can be <u>solved</u> and <u>verified</u> in <u>poly-nomial</u> time	ii) Solution to NP problems <u>cannot be solved or obtain</u> in polynomial time, but if the solution is given it can be verified in polynomial time
iii) P problems are <u>subset</u> of NP problems	iii) NP problems are <u>super-set</u> of P problems
iv) All P problems are <u>deterministic</u> in nature	iv) All NP problems are <u>non-deterministic</u> in nature
v) <u>Example</u> : selection sort, Linear search	v) <u>Example</u> : TSP, knapsack problem

NP Complete hard

NP Complete

i) NP hard problem (A) can be solved if and only if there is of NP complete problem (B) can be reducible into A in polynomial time.

i) NP complete problem can be solved by deterministic algorithm in polynomial time.

ii) to solve NP hard problem it must be in NP class.

ii) to solve NP complete problem, it must be in both.

iii) It is not a decision problem but it is a optimization problem.

iii) It is a decision problem.

iv) Example:

- i) Halting problem
- ii) Hamiltonian problem

iv) Example:

Determining whether a graph has a hamiltonian cycle or not.

* Polynomial problem Reduction (Reducability α)

i) To prove whether particular problem is NP complete or not, we use polynomial time reducability that means,

$\overset{\text{poly}}{A \rightarrow B}$ and $\overset{\text{poly}}{B \rightarrow C}$ then $\overset{\text{poly}}{A \rightarrow C}$

ii) Let, L_1 and L_2 be problems
 problem L_1 reduces to L_2 (also written as $L_1 \leq L_2$) if and only if there is a way to solve L_1 by a deterministic polynomial time algorithm using a deterministic algorithm that solves L_2 in polynomial time.

iii) This definition implies that if we have a polynomial time algorithm L_2 then we can solve L_1 in polynomial time.

iv) Reduction in NP complete problem takes one of these forms,

- Local Replacement
- Component Design

a) Local Replacement

In this reduction $A \leq B$ by dividing input to A in the form of components and then these components can be converted to components of B .

b) Component Design

In this reduction $A \leq B$ by building special components for input B that enforce properties required by A .

* Reduction in NP completeness

Every problems

in NP

↓
Circuit SAT

↓
CNF SAT

↓
3 SAT

↓
Vertex cover

↓
Clique set
cover

↓
Subset
sum

↓
Hamiltonian
cycle

↓
Knapsack

↓
TSP

Fig: Reduction in NP-completeness

* NP-complete problem

We are going to cover two

- 1) Vertex cover (Node cover)
- 2) 3 SAT (satisfiability)

Important

Steps to prove problem L_2 is NP Complete

Step 1:

To show that L_2 is NP hard

Step 2: An NP hard decision problem L_2 can be shown to be NP complete by exhibiting a polynomial time non-deterministic algorithm for L_2 (means to prove that it is also a part of NP)

Step 3: How to prove problem is NP hard

Step 1: pickup a problem L_1 already

Step 2: show how to obtain (in polynomial deterministic time) an instance I' of L_2 from any instance I of L_1 such that from the solution of I' we can determine (in polynomial deterministic time) the solution to instance I (L_1)

Step 3: Conclude from step 2 that $L_1 \leq L_2$

Step 4: conclude from step 1 and step 3 and transitivity theorem that L_2 is NP hard

* What examples to prove NP completeness of NP hard

* What is Satisfiability (SAT)?

→ Soln:

Let $x_1, x_2, \dots, x_n$ denotes boolean variable.

i) $\bar{x}_i$ denotes the negation of x_i . a Literal is either a variable and its negation

ii) A Formula in the propositional calculus is an expression that can be constructed using literals and operation AND, OR

iii) symbol 'V' denotes OR & symbol '∧' denotes AND

iv) CNF (Conjunctive Normal Form) is Formula like where each clause is connected using '∧' and literal using 'V'

v) Satisfiability problem is to determine whether the formula is true for some assignments of truth values or variables.

* What is 3-SAT problem?

3 SAT problem is a problem of determining the satisfiability of a CNF formula where each clause is limited to at most three literals.

3-SAT proved that 3-SAT

i) 3-SAT is NP. given an assignment we can just check that each clause is covered or we can verify the formula for given assignment.

ii) 3-SAT is NP-hard.

to prove this a reduction from SAT to 3-SAT must be provided

iii) We will transfer each clause independently based on its length.

iv) Reduction from SAT to 3-SAT

Suppose a clause contains k -literals

1) If $k=1$ (meaning $C_i = \{z_1\}$), we can add two new variables V_1 & V_2 and transform this into four clauses

$$(z_1 \vee V_1 \vee V_2) \wedge (z_1 \vee V_1 \vee \bar{V}_2) \wedge (z_1 \vee V_1 \vee \bar{V}_2) \wedge (z_1 \vee \bar{V}_1 \vee V_2)$$

2) If $k=2$ (meaning $C_i = \{z_1, z_2\}$), we can add one new variable V_1 and transform this into two new clauses

$$\{z_1, z_2, V_1\}$$

$$\{z_1, z_2, \bar{V}_1\}$$

3) If $k=3$ (meaning $C_i = \{z_1, z_2, z_3\}$), we move the clause as it is no need to add any new variable.

4) If $k \geq 3$ (means $C_i = \{z_1, z_2, z_3, \dots, z_k\}$), we

can add in $k-3$ new variables $(v_1, v_2, \dots, v_{k-3})$
and $k-2$ clauses

$$\{z_1, z_2, v_1\}$$

Example:

if $k=4$ then $k-3$ means $4-3=1$ new variables we have to introduced.

So, new variable will be v_1 and $\bar{v}_1$
and $k-2$ means $4-2=2$ clauses will be created.

given four variables are z_1, z_2, z_3, z_4 and new variables v_1 and $\bar{v}_1$

two clauses that will be created,

$$\{z_1, z_2, v_1\}$$

$$\{z_3, z_4, \bar{v}_1\}$$

- i) one of the z_i must be true for the original clause to be true.
- ii) It can be seen that if the original clause is satisfiable then its equivalent 3-SAT representation is also satisfiable too.

iii) As SAT instance can be reducible to SAT
(SAT $\leq$ 3-SAT)

so, we can conclude that 3-SAT is also NP-hard.

**** As 3-SAT is NP as well as 3-SAT is NP hard
(we can conclude) that 3-SAT is NP-complete.**

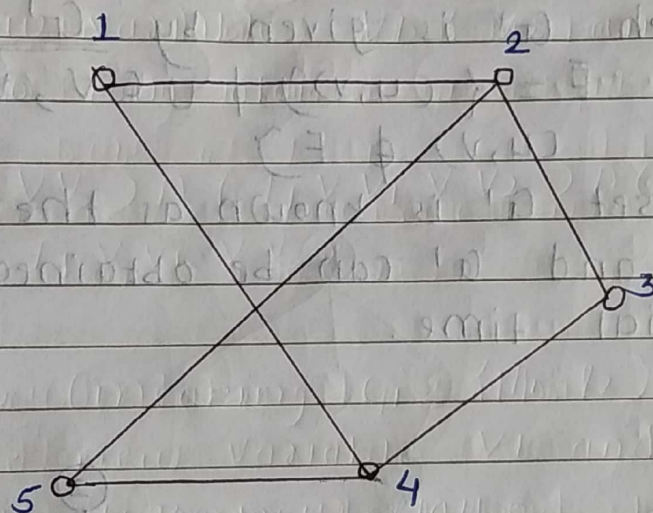
* Theorem :
 prove that vertex cover (node cover) is NP complete

* What is vertex cover problem ?

→ soln:

i) set S which is subset of V ($S \subseteq V$)
 is vertex cover for a graph $G = (V, E)$ if and only if all edges in E

ii) size $|S|$ of the cover is the number of vertices in S .



iii) in above example $S = \{2, 4\}$ is a vertex cover of size 2.

$S = \{1, 3, 5\}$ is a vertex cover of size 3.

iv) in vertex cover decision problem we are giving a graph G and an integer k , we are required to determine whether G has a node cover of size at most k .

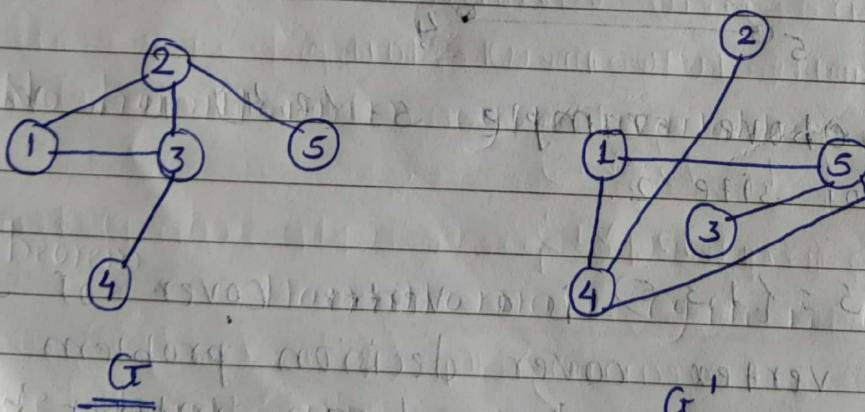
* 1) Theorem:

prove that Vertex Cover is NP Complete problem.

→ proof:

- i) let $G = \{V, E\}$ and k defined an instance of CDF (clique decision problem).
- ii) Assume that, $|V| = n$
- iii) we construct a graph G' has a node cover of such that G' has a node cover size at most $n-k$ if and only if G has a clique of size at least k .
- iv) Graph G' is given by $G' = (V, \bar{E})$ where $\bar{E} = \{(u, v) \mid u \in V, v \in V \text{ and } (u, v) \notin E\}$
- v) The set G' is known as the complement of G and G' can be obtained from G in polynomial time.

Example:



clique size = 3 $5-3=2$
 vertices = $\{1, 2, 3\}$ $\{4, 5\}$

Above Figure shows a graph G and its complement G' .

ii) In this fig, G' has vertex cover of $\{4, 5\}$, since every edge of G' is incident either on the node 4 or on the node 5.

iii) Thus G has a clique of size $5-2=3$, consisting of nodes $\{1, 2, 3\}$.

iv) since CNF satisfiability (3 SAT COP) and (COP $\propto$ vertex cover).

v) We can say that vertex cover is also NP hard (using transitivity theorem).

vi) Vertex cover is also in NP because we can non-deterministically choose a subset $(C \subseteq V)$ of size k and verify in polynomial time that C is a cover of G .

vii) As vertex cover is NP hard and also belongs to NP, we can conclude that vertex cover is NP-complete problem.

2) prove that Hamiltonian cycle problem is NP-complete.

Hamiltonian

Hamiltonian cycle or a circuit of a given graph is round trip path that starts at a specific path visit all nodes excluding the starting node in the graph.

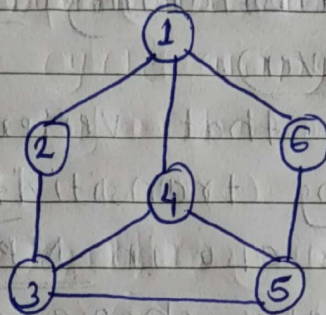
ii) A graph can have 0, 1 or multiple hamiltono-

Hamiltonian cycle in it.

iii) It applies to both directed and undirected graph.

iv) In travelling salesperson problem (TSP) a tour completed by a salesperson is a Hamiltonian cycle.

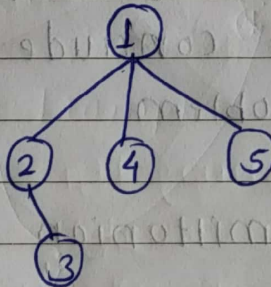
Example 1:



Graph has Hamiltonian cycle

Soln: (1, 2, 3, 4, 5, 6) OR (1, 6, 5, 4, 3, 2, 1)

Example 2:



No Hamiltonian cycle available

3) Theorem: Hamiltonian cycle is NP hard

→ We show SAT is reducible to DHC (SAT $\leq$ DHC) (directed Hamiltonian cycle)

proof:

Above Figure shows a graph G and its complement G' .

ii) In this Fig, G' has vertex cover of $\{4, 5\}$, since every edge of G' is incident either on the node 4 or on the node 5.

iii) Thus G has a clique of size $5-2=3$, consisting of nodes $\{1, 2, 3\}$.

iv) since CNF satisfiability ($3SAT \leq_{NP} COV$) and ($COV \leq_{NP} \text{vertex cover}$).

v) We can say that vertex cover is also NP hard (using transitivity theorem).

vi) vertex cover is also in NP because we can non-deterministically choose a subset $(C \subseteq V)$ of size k and verify in polynomial time that C is a cover of G .

vii) As vertex cover is NP hard and also belongs to NP, we can conclude that vertex cover is NP-complete problem.

2) prove that Hamiltonian cycle problem is NP-complete.

Hamiltonian

Hamiltonian cycle or a circuit of a given graph is round trip path that starts at a specific path visit all nodes excluding the starting node in the graph.

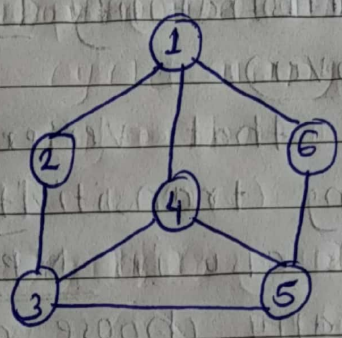
ii) A graph can have 0, 1 or multiple hamiltono-

Hamiltonian cycle in it.

iii) It applies to both directed and undirected graph.

iv) In travelling salesperson problem (TSP) a tour completed by a salesperson is a Hamiltonian cycle.

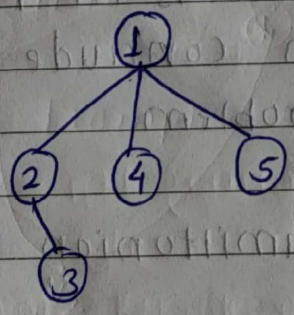
Example 1:



Graph has Hamiltonian cycle

Soln: (1, 2, 3, 4, 5, 6) OR (1, 6, 5, 4, 3, 2, 1)

Example 2:



No Hamiltonian cycle available

3) Theorem: Hamiltonian cycle is NP hard

→ We show SAT is reducible to DHC (SAT to DHC) (directed Hamiltonian cycle)

Proof:

i) let F be a propositional formula in CNF,
 ii) we will construct a directed graph G from formula F .

iii) Example:

$$F = C_1 \wedge C_2 \wedge C_3$$

$$C_1 = x_1 \vee \bar{x}_2 \vee x_4$$

$$C_2 = \bar{x}_1 \vee x_2 \vee x_3$$

$$C_3 = \bar{x}_1 \vee \bar{x}_3 \vee \bar{x}_4$$

$$F = (x_1 \vee \bar{x}_2 \vee x_4) \wedge (\bar{x}_1 \vee x_2 \vee x_3) \wedge (\bar{x}_1 \vee \bar{x}_3 \vee \bar{x}_4)$$

i) Assume that formula F has r no. of clauses $C_1, C_2, \dots, C_r$ & n variables $x_1, x_2, \dots, x_n$.

v) Draw an array with r rows and $2n$ columns

vi) row i denotes clause C_i

vii) Each variables x_i is represented by two adjacent columns, one for each of the literals x_i and $\bar{x}_i$.

Rules:

1) Insert the symbol $*$ into column x_i and row C_j if and only if x_i is a literal in C_j .
 same follow for $\bar{x}_i$.

2) Between each pair of columns x_i and $\bar{x}_i$ introduced two vertices u_i and v_i . u_i at top and v_i at bottom.

3) Draw two chains of edges upward from v_i to u_i .

one connecting together all $(*)$ in column x_i
and other connecting all $(*)$ in column $\bar{x}_i$

4) Now draw edges between u_i, v_{i+1}

5) ~~To complete the graph~~

As per the above we will draw the directed Hamiltonian cycle using given formula

Unit 1 (Revision)

